

Week 3 — Day 16 Manual (WSL)

Roman-Urdu Normalization and Dual-Query Retrieval (Merge Results)

Extension of Day 15: In the original warmup plan, Day 16 is Roman-Urdu normalization + dual-query retrieval (merge results). This comes immediately after the RAG endpoint and before guardrails, logging, and agent workflows.

1. Purpose of Day 16

By Day 15, you should already have a working RAG endpoint that:

- accepts a user question
- retrieves top-k chunks from Qdrant
- sends grounded context to an LLM
- returns an answer with sources

That is not enough for messy real-world queries.

Users do not always type clean English or clean Urdu. They often write in Roman Urdu, mix languages, misspell words, or use noisy phrasing. If your retrieval only works for clean text, then the system is fragile. Day 16 exists to make retrieval more tolerant by adding:

- Roman-Urdu query normalization
- dual-query retrieval
- merged search results across query variants

This day is retrieval-focused. It is not about new UI. It is not about agents. It is about making the retrieval stage less brittle.

2. Day 16 Scope

Today, you will extend your retrieval and RAG workflow so that one user query can produce two search queries:

1. the original query
2. a normalized query for cleaner retrieval

You will then:

- run retrieval for both
- merge the results
- deduplicate overlapping chunks
- send the merged context to the RAG endpoint

The goal is simple: better recall for noisy Roman-Urdu and mixed-language questions.

3. Outcomes and Deliverables

By the end of Day 16, your repository must contain:

- a Roman-Urdu normalization module
- dual-query retrieval logic
- merged and deduplicated retrieval results
- the RAG endpoint updated to use merged retrieval
- visible evidence that noisy queries retrieve better than before
- screenshots stored under:

docs/screenshots/week3-day16/

- a pull request merged into main

4. Preconditions

Before starting Day 16, the following must already work:

- your FastAPI app runs

- your Day 15 RAG endpoint works
- your Qdrant collection already contains chunked documents
- your retrieval module can search Qdrant
- your settings module is already environment-based
- your repository is still clean and using src/ layout

If Day 15 is not actually working, Day 16 will be fake progress. Fix that first.

5. Start in WSL

From Windows PowerShell:

```
wsl ~
```

In WSL:

```
cd ~/<your-repo>
source .venv/bin/activate
python -m pip install --upgrade pip
```

Create a Day 16 branch:

```
git checkout main
git pull
git checkout -b feature/week3-day16-roman-urdu-retrieval
```

6. Why Roman-Urdu Normalization Is Needed

Roman Urdu is inconsistent by nature. The same meaning may appear in several forms.

Examples:

- kya / kia
- hai / hy / he
- kyun / q k / kyu

- mujhe / muje / mjhe
- qdrant kya karta hai
- qdrant kia krta he

A retrieval system that depends on only one exact query form will miss relevant chunks too often. Day 16 is not trying to fully solve language understanding. It is adding a practical baseline so retrieval does not collapse under normal noisy usage.

7. Day 16 Design Strategy

Use a simple and practical baseline.

For every incoming question:

1. preserve the original query
2. generate a normalized version
3. search Qdrant with both
4. merge the results
5. deduplicate by chunk_id
6. keep the strongest results
7. pass those merged results into the RAG prompt

This is enough for Day 16. Do not overcomplicate it with fancy transliteration models unless you already have a strong reason.

8. Update the Repository Structure

By the end of Day 16, your structure should resemble:

```
README.md
Dockerfile
docker-compose.yml
pyproject.toml
.gitignore
.env.example
src/
  <package_name>/
```

```
__init__.py
api.py
settings.py
vector_store.py
retrieval.py
rag.py
llm_client.py
normalization.py
schema.py
tests/
scripts/
docs/
screenshots/
  week3-day15/
  week3-day16/
```

Notes

- normalization.py will hold Roman-Urdu cleanup logic
- retrieval.py will be extended for dual-query search
- rag.py will consume merged retrieval results
- api.py should remain thin

9. Add Retrieval Settings

Open:

```
src/<package_name>/settings.py
```

Add fields like these to your settings class if they are not already present:

```
from pydantic import Field
```

```
default_search_limit: int = Field(default=3)
dual_query_enabled: bool = Field(default=True,
validation_alias="DUAL_QUERY_ENABLED")
```

```
normalization_enabled: bool = Field(default=True,
validation_alias="NORMALIZATION_ENABLED")
merged_search_limit: int = Field(default=5, validation_alias="MERGED_SEARCH_LIMIT")
```

Update .env.example with safe placeholders:

```
DUAL_QUERY_ENABLED=true
NORMALIZATION_ENABLED=true
MERGED_SEARCH_LIMIT=5
```

Rules

- keep these values configurable
- do not hardcode behavior directly in the route
- merged retrieval count can be slightly larger than final RAG context count

10. Create the Normalization Module

Create:

```
src/<package_name>/normalization.py
```

Paste:

```
from __future__ import annotations
```

```
import re
```

```
ROMAN_URDU_REPLACEMENTS = {
    "kia": "kya",
    "kya": "kya",
    "kr": "kar",
    "krta": "karta",
    "krti": "karti",
    "krna": "karna",
```

```
"he": "hai",
"hy": "hai",
"hn": "hain",
"mje": "mujhe",
"muje": "mujhe",
"mjhe": "mujhe",
"qdrnt": "qdrant",
"retrival": "retrieval",
"maloomat": "malumat",
"javab": "jawab",
"btado": "bata do",
"btao": "batao",
}
```

```
def basic_cleanup(text: str) -> str:
    text = text.strip().lower()
    text = re.sub(r"\s+", " ", text)
    return text
```

```
def normalize_roman_urdu(text: str) -> str:
    cleaned = basic_cleanup(text)
    words = cleaned.split()

    normalized_words = []
    for word in words:
        normalized_words.append(ROMAN_URDU_REPLACEMENTS.get(word, word))

    normalized = " ".join(normalized_words)
    normalized = re.sub(r"\s+", " ", normalized).strip()
    return normalized
```

Important Note

This is not a perfect Roman-Urdu language model. It is a baseline normalization layer. That is enough for this day.

11. Extend the Retrieval Module for Dual-Query Search

Open:

```
src/<package_name>/retrieval.py
```

Add or refactor the module so it can:

- search using one query
- search using two query variants
- merge and deduplicate results

A practical version looks like this:

```
from __future__ import annotations
```

```
from functools import lru_cache
```

```
from sentence_transformers import SentenceTransformer
```

```
from <package_name>.normalization import normalize_roman_urdu
```

```
from <package_name>.settings import settings
```

```
from <package_name>.vector_store import get_qdrant_client
```

```
@lru_cache(maxsize=1)
```

```
def get_embedding_model() -> SentenceTransformer:
```

```
    return SentenceTransformer(settings.embedding_model_name)
```

```
def search_chunks(query: str, limit: int) -> list[dict]:
```

```
    model = get_embedding_model()
```

```
    client = get_qdrant_client()
```

```
    query_vector = model.encode(query, normalize_embeddings=True).tolist()
```

```
    results = client.search(
```

```
        collection_name=settings.qdrant_collection_name,
```

```
        query_vector=query_vector,
```



```

        limit=limit,
    )

    formatted_results: list[dict] = []
    for point in results:
        payload = point.payload or {}
        formatted_results.append(
            {
                "score": float(point.score),
                "doc_id": payload.get("doc_id", ""),
                "chunk_id": payload.get("chunk_id", ""),
                "title": payload.get("title", ""),
                "language": payload.get("language", ""),
                "source": payload.get("source", ""),
                "chunk_index": payload.get("chunk_index", 0),
                "text": payload.get("text", ""),
            }
        )

```

```

    return formatted_results

```

```

def merge_results(result_sets: list[list[dict]], final_limit: int) -> list[dict]:
    best_by_chunk_id: dict[str, dict] = {}

```

```

    for result_set in result_sets:
        for item in result_set:
            chunk_id = item["chunk_id"]
            if chunk_id not in best_by_chunk_id:
                best_by_chunk_id[chunk_id] = item
            else:
                if item["score"] > best_by_chunk_id[chunk_id]["score"]:
                    best_by_chunk_id[chunk_id] = item

```

```

    merged = list(best_by_chunk_id.values())
    merged.sort(key=lambda x: x["score"], reverse=True)
    return merged[:final_limit]

```

```

def dual_query_search(query: str, limit: int) -> dict:

```

```

original_query = query.strip()
normalized_query = normalize_roman_urdu(query)

original_results = search_chunks(original_query, limit)
normalized_results = search_chunks(normalized_query, limit)

merged_results = merge_results(
    [original_results, normalized_results],
    final_limit=settings.merged_search_limit,
)

return {
    "original_query": original_query,
    "normalized_query": normalized_query,
    "results": merged_results,
}

```

Important: Replace <package_name> with your actual package name.

12. Update the RAG Module to Use Dual Retrieval

Open:

src/<package_name>/rag.py

Update it so that the RAG flow uses dual-query retrieval instead of single-query retrieval.

A practical version is:

```

from __future__ import annotations

from <package_name>.llm_client import generate_answer_from_prompt
from <package_name>.retrieval import dual_query_search

def build_context_block(results: list[dict]) -> str:
    parts: list[str] = []

```

```
for idx, item in enumerate(results, start=1):
```

```
    parts.append(
```

```
        f"[Source {idx}]\n"
```

```
        f"doc_id: {item['doc_id']}\n"
```

```
        f"chunk_id: {item['chunk_id']}\n"
```

```
        f"title: {item['title']}\n"
```

```
        f"text: {item['text']}\n"
```

```
    )
```

```
return "\n".join(parts)
```

```
def build_rag_prompt(question: str, results: list[dict]) -> str:
```

```
    context = build_context_block(results)
```

```
    return f"""
```

Answer the question using only the context below.

Question:

{question}

Context:

{context}

Rules:

- Use only the provided context.

- Do not invent facts.

- If the answer is not supported by the context, say that clearly.

- Mention supporting sources using the source numbers.

```
""").strip()
```

```
def answer_with_rag(question: str, limit: int) -> dict:
```

```
    retrieval_payload = dual_query_search(question, limit)
```

```
    results = retrieval_payload["results"]
```

```
    prompt = build_rag_prompt(question, results)
```

```
    answer = generate_answer_from_prompt(prompt)
```

```
return {
    "question": question,
    "normalized_query": retrieval_payload["normalized_query"],
    "answer": answer,
    "sources": results,
}
```

13. Update the Response Schemas

Open:

```
src/<package_name>/schema.py
```

Extend your Day 15 response model so it includes the normalized query.

Add or update:

```
from __future__ import annotations
```

```
from pydantic import BaseModel, Field
```

```
class RagRequest(BaseModel):
```

```
    question: str = Field(min_length=1, description="The user question.")
```

```
    limit: int = Field(default=3, ge=1, le=10)
```

```
class RagSourceItem(BaseModel):
```

```
    score: float
```

```
    doc_id: str
```

```
    chunk_id: str
```

```
    title: str
```

```
    language: str
```

```
    source: str
```

```
    chunk_index: int
```

```
    text: str
```

```
class RagResponse(BaseModel):
    question: str
    normalized_query: str
    answer: str
    sources: list[RagSourceItem]
```

Why this matters

You need to see what the system actually normalized the query into. Otherwise, debugging Day 16 becomes guesswork.

14. Update the RAG Endpoint

Open:

```
src/<package_name>/api.py
```

Update the /rag route so it returns the normalized query.

Example:

```
from __future__ import annotations

from fastapi import FastAPI, HTTPException

from <package_name>.rag import answer_with_rag
from <package_name>.schema import RagRequest, RagResponse

app = FastAPI(title="AI Retrieval and RAG API")

@app.post("/rag", response_model=RagResponse)
def rag_endpoint(payload: RagRequest) -> RagResponse:
    try:
        result = answer_with_rag(payload.question, payload.limit)
```

```

except Exception as exc:
    raise HTTPException(status_code=500, detail=f"RAG failed: {exc}") from exc

return RagResponse(
    question=result["question"],
    normalized_query=result["normalized_query"],
    answer=result["answer"],
    sources=result["sources"],
)

```

Do not overwrite your app carelessly. Merge this into the existing structure.

15. Add a Small Query Debug Script

Create:

scripts/day16_dual_query_debug.py

Paste:

```

from __future__ import annotations

from <package_name>.retrieval import dual_query_search

def main() -> None:
    queries = [
        "qdrant kia karta he",
        "docker compose kia krta he",
        "mujhe retrieval ka jawab do",
    ]

    for query in queries:
        result = dual_query_search(query, limit=3)
        print("=" * 70)
        print(f"Original Query: {result['original_query']}")
        print(f"Normalized Query: {result['normalized_query']}")

```

```

print("-" * 70)
for idx, item in enumerate(result["results"], start=1):
    print(
        {
            "rank": idx,
            "score": item["score"],
            "chunk_id": item["chunk_id"],
            "title": item["title"],
            "text": item["text"],
        }
    )

if __name__ == "__main__":
    main()

```

Replace <package_name> with the real package name.

Run it with:

```

export PYTHONPATH=src
python scripts/day16_dual_query_debug.py

```

16. Run the API

Because your project uses src/ layout:

```

export PYTHONPATH=src
fastapi dev src/<package_name>/api.py

```

Or with Uvicorn:

```

export PYTHONPATH=src
uvicorn <package_name>.api:app --reload

```

Replace <package_name> with the actual package name.

17. Test the Updated RAG Endpoint

Open Swagger UI:

<http://127.0.0.1:8000/docs>

Try a clean query first:

```
{  
  "question": "What does Qdrant do in this system?",  
  "limit": 3  
}
```

Then try noisy Roman-Urdu queries:

```
{  
  "question": "qdrant kia karta he",  
  "limit": 3  
}  
{  
  "question": "docker compose kia krta he",  
  "limit": 3  
}  
{  
  "question": "retrieval sy jawab kesay milta he",  
  "limit": 3  
}
```

Expected Behavior

The response should now include:

- the original question
- the normalized query
- the answer
- merged source results

The exact output will vary. The important part is that retrieval should become more tolerant to noisy phrasing.

18. Add a Basic Test File

Create:

```
tests/test_day16_dual_query.py
```

Paste:

```
from <package_name>.normalization import normalize_roman_urdu
```

```
def test_normalize_roman_urdu_changes_common_forms() -> None:
    assert normalize_roman_urdu("qdrant kia karta he") == "qdrant kya karta hai"
```

```
def test_normalize_roman_urdu_keeps_clean_text_stable() -> None:
    assert normalize_roman_urdu("what does qdrant do") == "what does qdrant do"
```

Replace <package_name> with the real one.

Note

This is not enough testing for a serious system. It is enough for the day's baseline. Day 17 will add more control and logging.

19. Create the Evidence Folder

Create:

```
mkdir -p docs/screenshots/week3-day16
touch docs/screenshots/week3-day16/.gitkeep
```

Required screenshots:

1. 01-normalization-code.png

normalization.py open in the editor

2. 02-dual-query-debug.png

terminal output from day16_dual_query_debug.py

3. 03-rag-noisy-query-request.png

Swagger request for a noisy Roman-Urdu query

4. 04-rag-noisy-query-response.png

successful /rag response showing normalized_query

5. 05-result-merge-proof.png

terminal or code view showing merged/deduplicated retrieval logic

6. 06-pytest-pass.png

terminal output showing Day 16 tests pass

7. 07-git-status-clean.png

terminal output showing only intended tracked files

20. Update the Root README

Your README.md must now reflect Day 16 progress.

Update these sections:

Current Scope

Add that the project now supports:

- RAG answers over retrieved chunks
- Roman-Urdu query normalization

- dual-query retrieval
- merged retrieval results for noisy inputs

Repository Structure

Include:

- normalization.py

Progress by Day

Update to include:

- Day 15: RAG endpoint v1
- Day 16: Roman-Urdu normalization and dual-query retrieval

Next Steps

Your next steps should now point toward:

- confidence scoring
- refusal / clarify behavior
- logging to Postgres
- guardrails for weak retrieval

This matches the original warmup plan sequence, where Day 17 is guardrails after Day 16.

21. Git Workflow

Stage Files

```
git add \  
README.md \  
.env.example \  
src/ \  
tests/ \  
scripts/ \  
docs/screenshots/week3-day16/.gitkeep
```

Commit

```
git commit -m "Add Roman-Urdu normalization and dual-query retrieval"
```

Push

```
git push -u origin feature/week3-day16-roman-urdu-retrieval
```

Open a Pull Request

- Base: main
- Compare: feature/week3-day16-roman-urdu-retrieval

PR Title:

Day 16: Roman-Urdu normalization and dual-query retrieval

PR Description must include:

- how to run the API
- how to call /rag with a noisy Roman-Urdu question
- how to run the debug script:

```
export PYTHONPATH=src  
python scripts/day16_dual_query_debug.py
```

- evidence path:

docs/screenshots/week3-day16/

22. Completion Checklist

Before marking Day 16 complete, confirm all of the following:

- Roman-Urdu normalization logic exists

- retrieval can search using both original and normalized queries
- merged results are deduplicated by chunk
- the RAG endpoint uses merged retrieval results
- the response includes `normalized_query`
- noisy Roman-Urdu queries work better than before
- screenshots are stored under `docs/screenshots/week3-day16/`
- the root `README.md` reflects Day 16 progress
- the pull request has been merged into main

23. Practical Notes

Keep these points in mind:

- This normalization layer is a baseline, not a real transliteration system. Do not pretend it is more than that.
- If retrieval quality is still poor, the problem may be chunk quality or missing relevant documents, not only query phrasing.
- If normalization changes clean queries into worse ones, your replacement rules are too aggressive.
- If merged retrieval produces duplicates, your deduplication logic is weak.
- If the answer is still hallucinated, Day 17 guardrails are not optional. They are necessary.

24. What Day 16 Prepares You For

After completing Day 16, you should be ready for:

- confidence-based refusal
- clarification behavior for weak questions
- logging retrieval and answer traces to Postgres
- stronger control over unanswerable questions

That is exactly the next planned step, because Day 17 in the warmup plan is guardrails (confidence, refusal/clarify, logging to Postgres).